

SciPy Stats

The `scipy.stats` contains a large number of statistics, probability distributions functions. The list of statistics functions can be obtained by `info(stats)`.

Descriptive Statistics

The descriptive statistics describe the values of observation in a variable. There are various stats such as Min, Max, and Variance, that take the Numpy array as input and returns the particular results. Some essential functions provide by `scipy.stats` package are described in the following image.

In [2]:

```
from scipy import stats
import numpy as np
x = np.array([1,2,3,4,5,6,7,8,9])
print(x.max(),x.min(),x.mean(),x.var())
```

```
9 1 5.0 6.666666666666667
```

In [3]:

```
stats.describe(x)
```

Out[3]:

```
DescribeResult(nobs=9, minmax=(1, 9), mean=5.0, variance=7.5, skewness=0.0,
kurtosis=-1.2300000000000002)
```

In [4]:

```
stats.gmean([1, 2, 3, 4, 5, 6, 7])
```

Out[4]:

```
3.3800151591412964
```

In [5]:

```
a = np.array([[6, 8, 3, 0],
              [3, 2, 1, 7],
              [8, 1, 8, 4],
              [5, 3, 0, 5],
              [4, 7, 5, 9]])
```

In [6]:

```
stats.mode(a, axis=None)
```

Out[6]:

```
ModeResult(mode=array([3]), count=array([3]))
```

In [7]:

```
?stats
```

scipy.stats.tmean(a, limits=None, inclusive=(True, True), axis=None) **[source]**

Compute the trimmed mean.

This function finds the arithmetic mean of given values, ignoring values outside the given limits.

In [21]:

```
x = np.arange(20)
stats.tmean(x)
```

Out[21]:

9.5

scipy.stats.tvar(a, limits=None, inclusive=(True, True), axis=0, ddof=1) **[source]**

Compute the trimmed variance.

This function computes the sample variance of an array of values, while ignoring values which are outside of given limits.

In [20]:

```
x = np.arange(20)
stats.tvar(x)
```

Out[20]:

35.0

In [22]:

```
import scipy

input_array = scipy.mean([2, 12, 9, 37, 20, 10, 4, 27])

print("Arithmetic Mean of the input array :", input_array)
```

Arithmetic Mean of the input array : 15.125

C:\Users\admin\AppData\Local\Temp\ipykernel_8476\3177645877.py:3: Deprecatio
nWarning: scipy.mean is deprecated and will be removed in SciPy 2.0.0, use n
umpy.mean instead

```
input_array = scipy.mean([2, 12, 9, 37, 20, 10, 4, 27])
```

In [24]:

```
from scipy import mean

arr1 = [[2, 12, 9],
        [37, 20, 10],
        [4, 27, 13],
        [9, 12, 26]]

print("Arithmetic Mean with axis = 0 : ",
      mean(arr1, axis = 0))
print("Arithmetic Mean with axis = 1 : ",
      mean(arr1, axis = 1))
```

```
Arithmetic Mean with axis = 0 : [13.   17.75 14.5 ]
Arithmetic Mean with axis = 1 : [ 7.66666667 22.33333333 14.66666667 15.666
66667]
```

C:\Users\admin\AppData\Local\Temp\ipykernel_8476\2205877764.py:9: Deprecatio
nWarning: scipy.mean is deprecated and will be removed in SciPy 2.0.0, use n
umpy.mean instead

```
mean(arr1, axis = 0))
C:\Users\admin\AppData\Local\Temp\ipykernel_8476\2205877764.py:11: Deprecati  
onWarning: scipy.mean is deprecated and will be removed in SciPy 2.0.0, use  
numpy.mean instead
mean(arr1, axis = 1))
```

In [27]:

```
scipy.median(arr1, axis=0)
```

C:\Users\admin\AppData\Local\Temp\ipykernel_8476\2951359389.py:1: Deprecatio
nWarning: scipy.median is deprecated and will be removed in SciPy 2.0.0, use
numpy.median instead

```
scipy.median(arr1, axis=0)
```

Out[27]:

```
array([ 6.5, 16. , 11.5])
```

scipy.stats.nanmedian(array, axis=0) function calculates the median by ignoring the Nan (not a number) values of the array elements along the specified axis of the array.

In [28]:

```
arr1 = [1, 3, np.nan, 27, 2, 5]

print("median using nanmedian :", scipy.nanmedian(arr1))

print("median without handling nan value :", scipy.median(arr1))
```

```
median using nanmedian : 3.0
median without handling nan value : nan
```

```
C:\Users\admin\AppData\Local\Temp\ipykernel_8476\4168307007.py:3: Deprecatio
nWarning: scipy.nanmedian is deprecated and will be removed in SciPy 2.0.0,
use numpy.nanmedian instead
```

```
print("median using nanmedian :", scipy.nanmedian(arr1))
C:\Users\admin\AppData\Local\Temp\ipykernel_8476\4168307007.py:5: Deprecatio
nWarning: scipy.median is deprecated and will be removed in SciPy 2.0.0, use
numpy.median instead
print("median without handling nan value :", scipy.median(arr1))
```

scipy.stats.variation(a, axis=0, nan_policy='propagate', ddof=0, *, keepdims=False)[source]

Compute the coefficient of variation.

The coefficient of variation is the standard deviation divided by the mean. This function is equivalent to:

`np.std(x, axis=axis, ddof=ddof) / np.mean(x)` The default for `ddof` is 0, but many definitions of the coefficient of variation use the square root of the unbiased sample variance for the sample standard deviation, which corresponds to `ddof=1`.

The function does not take the absolute value of the mean of the data, so the return value is negative if the mean is negative.

Parameters `aarray_like` Input array.

`axis`int or None, optional Axis along which to calculate the coefficient of variation. Default is 0. If None, compute over the whole array `a`.

`nan_policy`{'propagate', 'raise', 'omit'}, optional Defines how to handle when input contains nan. The following options are available:

'propagate': return nan

'raise': raise an exception

'omit': perform the calculation with nan values omitted

The default is 'propagate'.

`ddof`int, optional Gives the "Delta Degrees Of Freedom" used when computing the standard deviation. The divisor used in the calculation of the standard deviation is $N - \text{ddof}$, where N is the number of elements. `ddof` must be less than N ; if it isn't, the result will be nan or inf, depending on N and the values in the array. By default `ddof` is zero for backwards compatibility, but it is recommended to use `ddof=1` to ensure that the sample standard deviation is computed as the square root of the unbiased sample variance.

`keepdims`bool, optional If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns variationndarray The calculated variation along the requested axis.

In [9]:

```
from scipy.stats import variation
variation([1, 2, 3, 4, 5], ddof=0)
```

Out[9]:

0.47140452079103173

In [13]:

```
x = np.array([[ 10.0, np.nan, 11.0, 19.0, 23.0, 29.0, 98.0],
              [ 29.0,  30.0, 32.0, 33.0, 35.0, 56.0, 57.0],
              [np.nan, np.nan, 12.0, 13.0, 16.0, 16.0, 17.0]])
variation(x, axis=1, ddof=1, nan_policy='omit')
```

Out[13]:

```
masked_array(data=[1.0510936133064064, 0.3142898569582722,
                  0.14648299586945135],
             mask=[False, False, False],
             fill_value=1e+20)
```

scipy.stats.tstd(a, limits=None, inclusive=(True, True), axis=0, ddof=1) **[source]**

Compute the trimmed sample standard deviation.

This function finds the sample standard deviation of given values, ignoring values outside the given limits.
Parameters aarray_like Array of values.

limits: None or (lower limit, upper limit), optional Values in the input array less than the lower limit or greater than the upper limit will be ignored. When limits is None, then all values are used. Either of the limit values in the tuple can also be None representing a half-open interval. The default value is None.

inclusive:(bool, bool), optional A tuple consisting of the (lower flag, upper flag). These flags determine whether values exactly equal to the lower or upper limits are included. The default value is (True, True).

axis: int or None, optional Axis along which to operate. Default is 0. If None, compute over the whole array a.

ddof: int, optional Delta degrees of freedom. Default is 1.

In [42]:

```
x = np.arange(20)
stats.tstd(x)
```

Out[42]:

5.916079783099616

In [14]:

```
from scipy.stats import skew  
skew([1, 2, 3, 4, 5])
```

Out[14]:

0.0

In [15]:

```
skew([2, 8, 0, 4, 1, 9, 9, 0])
```

Out[15]:

0.2650554122698573

In []: